Part I: Core Concepts

1.1 Functions

Concept: Passing Functions as Arguments

In C++, functions can be passed as arguments to other functions using three mechanisms:

- **Function pointers** — the classical C approach.
- **Function objects (functors)** — classes that overload `operator()`.
- **Lambda expressions** — anonymous inline functions (C++11 and later).

All three can be unified under `std::function<ReturnType(ArgTypes)>`, a type-erased callable wrapper from `<functional>`.

Example 1 — Function Pointers

```
#include <iostream>
using namespace std;

int add(int a, int b)      { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }

// Function pointer type: pointer to a function taking two ints, returning int
typedef int (*BinaryOp)(int, int);

int applyOp(int x, int y, BinaryOp op) { return op(x, y); }

int main() {
    BinaryOp ops[]   = { add, subtract, multiply };
    const char* names[] = { "add", "subtract", "multiply" };

    for (int i = 0; i < 3; i++)
        cout << names[i] << "(10, 3) = " << applyOp(10, 3, ops[i]) << "\n";
}
```

Expected output:

```
add(10, 3) = 13
subtract(10, 3) = 7
multiply(10, 3) = 30
```

1.2 Function Objects (Functors)

Concept: Functors and `std::function`

A **functor** is any class that overloads `operator()`. Unlike plain function pointers, functors can carry **state**.

`std::function<R(Args...)>` can hold any callable — function pointer, functor, or lambda — making it ideal for callbacks and higher-order functions.

Example 2 — Stateful Functor (Accumulator)

```
#include <iostream>
#include <functional>
using namespace std;

class Accumulator {
    int total = 0;
public:
    int operator()(int x) { total += x; return total; }
    int getTotal() const  { return total; }
};

int applyAndPrint(int val, function<int(int)> f) {
    int result = f(val);
    cout << "Running total: " << result << "\n";
    return result;
}

int main() {
    Accumulator acc;
    applyAndPrint(10, ref(acc));
    applyAndPrint(25, ref(acc));
    applyAndPrint(5,  ref(acc));
    cout << "Final total: " << acc.getTotal() << "\n";
}
```

Expected output:

```
Running total: 10
Running total: 35
Running total: 40
Final total: 40
```

1.3 Lambda Expressions

Concept: Lambda Syntax and Capture Lists

```
[capture](parameters) -> return_type { body }
```

- **Capture list** controls which outer variables the lambda can access:
  - `[]` — capture nothing.
  - `[x]` — capture `x` by value.
  - `[&x]` — capture `x` by reference.
  - `[=]` — capture all used variables by value.
  - `[&]` — capture all used variables by reference.
  - `[=, &x]` — capture all by value except `x` by reference.
- Add `mutable` to allow modifying value-captured variables inside the lambda.
- **Generic lambdas** (C++14): use `auto` as a parameter type.

Example 3 — Lambda with Capture and `mutable`

```cpp
#include <iostream>
using namespace std;

int main() {
    int base = 100;

    // Capture by value (copy) - 'base' inside lambda is independent
    auto addBase = [base](int x) mutable {
        base += x;          // modifies the *copy*, not the outer 'base'
        return base;
    };

    // Capture by reference - modifies the outer 'base' directly
    auto addBaseRef = [&base](int x) {
        base += x;
        return base;
    };

    cout << "By value:      " << addBase(10) << "\n";  // 110
    cout << "Outer base:    " << base        << "\n";  // 100 (unchanged)
    cout << "By reference:  " << addBaseRef(10) << "\n"; // 110
    cout << "Outer base:    " << base        << "\n";  // 110 (changed)
}
```

Expected output:

```
By value:      110
Outer base:    100
```

```
By reference: 110
Outer base:   110
```

Part II: Map, Filter, and Reduce

2.1 Map — `std::transform`

Concept: `std::transform` (Map)

`std::transform` applies a function to every element of a range and writes the result to an output range. It is the C++ equivalent of a functional *map* operation.

```
#include <algorithm>
// Unary form: apply f to each element of [first, last)
std::transform(first, last, output_first, unary_f);

// Binary form: combine two ranges element-wise
std::transform(first1, last1, first2, output_first, binary_f);
```

Example 4 — Unary and Binary Transform

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4, 5};

    // Unary: square each element
    vector<int> squares(v.size());
    transform(v.begin(), v.end(), squares.begin(),
              [](int x) { return x * x; });

    cout << "Squares: ";
    for (int x : squares) cout << x << " ";
    cout << "\n";

    // Binary: element-wise sum of two vectors
    vector<int> w = {10, 20, 30, 40, 50};
    vector<int> sums(v.size());
    transform(v.begin(), v.end(), w.begin(), sums.begin(),
              [](int a, int b) { return a + b; });

    cout << "Sums:    ";
    for (int x : sums) cout << x << " ";
    cout << "\n";
}
```

Expected output:

```
Squares: 1 4 9 16 25
Sums:    11 22 33 44 55
```

2.2 Filter — `std::copy_if` and `std::remove_if`

Concept: `std::copy_if` and Erase-Remove Idiom

- `std::copy_if` copies elements satisfying a predicate to an output range.
- `std::remove_if` moves non-matching elements to the front and returns an iterator to the new logical end. Combine with `vector::erase` to actually shrink the container (*erase-remove idiom*).

Example 5 — Filter Even Numbers

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    // copy_if: keep only even numbers
    vector<int> evens;
    copy_if(v.begin(), v.end(), back_inserter(evens),
            [](int x) { return x % 2 == 0; });

    cout << "Evens: ";
    for (int x : evens) cout << x << " ";
    cout << "\n";

    // Erase-remove idiom: remove odd numbers in-place
    v.erase(remove_if(v.begin(), v.end(),
                      [](int x) { return x % 2 != 0; }),
            v.end());

    cout << "After remove_if: ";
    for (int x : v) cout << x << " ";
    cout << "\n";
}
```

Expected output:

```
Evens: 2 4 6 8 10
After remove_if: 2 4 6 8 10
```

2.3 Reduce — `std::accumulate`

Concept: `std::accumulate` (Reduce)

`std::accumulate` collapses a range into a single value by repeatedly applying a binary operation, starting from an initial value.

```
#include <numeric>
T result = std::accumulate(first, last, init, binary_op);
```

- Default binary op is `+`, so the default computes the sum.
- Supply a lambda or functor for other reductions (product, min, max, string concat, ...).
- C++17 introduces `std::reduce` with parallel execution policies.

Example 6 — Sum, Product, and String Concatenation

```
#include <iostream>
#include <vector>
#include <numeric>
#include <string>
using namespace std;

int main() {
    vector<int> nums = {1, 2, 3, 4, 5};

    int sum = accumulate(nums.begin(), nums.end(), 0);
    cout << "Sum:     " << sum << "\n";

    int product = accumulate(nums.begin(), nums.end(), 1,
                             [](int a, int b) { return a * b; });
    cout << "Product: " << product << "\n";

    vector<string> words = {"Hello", " ", "from", " ", "C++!"};
    string sentence = accumulate(words.begin(), words.end(), string{});
    cout << "Sentence: " << sentence << "\n";

    // Compute mean using accumulate
    double mean = static_cast<double>(sum) / nums.size();
    cout << "Mean:    " << mean << "\n";
}
```

Expected output:

```
Sum:     15
Product: 120
Sentence: Hello from C++!
Mean:    3
```

Part III: Composition and Advanced Topics

3.1 Composing Map, Filter, and Reduce

Concept: Functional Pipeline

Real-world data processing pipelines chain map, filter, and reduce together. Given a collection of raw data, a typical pipeline:

1. **Filter** — discard irrelevant records.
2. **Map** (transform) — extract or compute the relevant field.
3. **Reduce** (accumulate) — aggregate into a result.

Example 7 — Pipeline: Total Sale of High-Value Items

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
using namespace std;

struct Product { string name; double price; int qty; };

int main() {
    vector<Product> products = {
        {"Laptop",   1200.0, 3},
        {"Mouse",      25.0, 10},
        {"Monitor",  400.0, 5},
        {"Keyboard",  60.0, 8},
        {"Tablet",   500.0, 2}
    };

    // Step 1 --- Filter: keep only items priced >= 100
    vector<Product> expensive;
    copy_if(products.begin(), products.end(), back_inserter(expensive),
            [](const Product& p) { return p.price >= 100.0; });

    // Step 2 --- Map: compute revenue per product
    vector<double> revenues(expensive.size());
    transform(expensive.begin(), expensive.end(), revenues.begin(),
              [](const Product& p) { return p.price * p.qty; });

    // Step 3 --- Reduce: total revenue
    double total = accumulate(revenues.begin(), revenues.end(), 0.0);

    cout << "High-value products:\n";
    for (const auto& p : expensive)
        cout << "  " << p.name << " ($" << p.price << " x " << p.qty << ")\n";
    cout << "Total revenue: $" << total << "\n";
}
```

Expected output:

```
High-value products:
  Laptop ($1200 x 3)
  Monitor ($400 x 5)
  Tablet ($500 x 2)
Total revenue: $6600
```

3.2 `std::bind` and Function Composition

Concept: `std::bind` and Partial Application

`std::bind` creates a new callable by binding some (or all) arguments of an existing function. `std::placeholders::_1`, `_2`, ... mark positions left unbound.

```
#include <functional>
auto add5 = bind(add, placeholders::_1, 5);  // add5(x) == add(x, 5)
```

Function composition chains two functions so the output of the first becomes the input of the second. C++ has no built-in compose, but it is easy to write as a template.

Example 8 — `std::bind` and a Generic Compose Helper

```
#include <iostream>
#include <functional>
#include <vector>
#include <algorithm>
using namespace std;

bool isGreaterThan(int x, int threshold) { return x > threshold; }

// Generic function composition: compose(f, g)(x) == f(g(x))
template <typename F, typename G>
auto compose(F f, G g) {
    return [=](auto x) { return f(g(x)); };
}

int main() {
    // Partial application: fix threshold = 5
    auto greaterThan5 = bind(isGreaterThan, placeholders::_1, 5);

    vector<int> v = {3, 6, 1, 8, 5, 9, 2};
    vector<int> result;
    copy_if(v.begin(), v.end(), back_inserter(result), greaterThan5);

    cout << "Greater than 5: ";
    for (int x : result) cout << x << " ";
    cout << "\n";

    // Function composition: double then add 3
    auto doubleIt = [](int x) { return x * 2; };
```

```
    auto add3      = [](int x) { return x + 3; };
    auto doubleThenAdd3 = compose(add3, doubleIt);

    cout << "compose(add3, double)(7) = " << doubleThenAdd3(7) << "\n";
}
```

Expected output:

```
Greater than 5: 6 8 9
compose(add3, double)(7) = 17
```

Quick Reference

Feature	Header	Brief Description
Function pointer	—	Classic callable; no state
Functor	—	Class with `operator()`; can carry state
`std::function`	`<functional>`	Type-erased callable wrapper
Lambda	—	Inline anonymous function; closure over local scope
`std::transform`	`<algorithm>`	Map: apply function element-wise
`std::copy_if`	`<algorithm>`	Filter: copy matching elements
`std::remove_if`	`<algorithm>`	Filter in-place (use with `erase`)
`std::accumulate`	`<numeric>`	Reduce: fold range into single value
`std::reduce`	`<numeric>`	Reduce (C++17, parallel-friendly)
`std::bind`	`<functional>`	Partial application / argument binding

Lab Exercises

Exercise 1

[**Function Pointers**] Write a calculator program that uses an **array of function pointers** to perform four operations: addition, subtraction, multiplication, and division.

- Define four separate functions for each operation.
- Store them in an array of function pointer type `double (*ops[])(double, double)`.
- Write a helper `double compute(double a, double b, double (*op)(double, double))` that invokes the chosen operation.
- In `main()`, loop over all four operations for inputs `a = 15.0`, `b = 4.0` and print each result.
- Use `using` (or `typedef`) to give the function pointer type a clean alias before defining the array.

Exercise 2

[**Functors and `std::function`**] Implement a **stateful Multiplier functor** and a standalone **event dispatcher**.

- Create a class `Multiplier` whose constructor takes a `double factor`. `operator()(double x)` returns `x * factor` and updates a private `callCount` member. Add a `getCount()` accessor.
- Write a function `void dispatch(double val, vector<function<double(double)>>& handlers)` that calls every handler in the vector with `val` and prints the result.
- In `main()`, create `Multiplier` objects for factors 2, 3, and 5. Register them (via `ref`) into the vector and dispatch the value `6.0`. After dispatch, print the call count of each multiplier.

Exercise 3

[**Lambda Expressions and Capture Lists**] Demonstrate all four capture modes using a practical scenario.

- Declare variables `int min = 10, max = 50, count = 0`.
- Lambda `inRange`: captures `min` and `max` by value; returns `true` if the argument is in `[min, max]`.
- Lambda `countInRange`: captures `count` by reference and `inRange` by value; increments `count` each time it is called with a value inside the range, then returns `count`.
- Lambda `scaleAndCheck`: generic lambda (C++14) that takes `auto x` and `auto factor`, returns the scaled value only if it is still in range, otherwise returns `-1`.
- Test with at least five values and print the running count after each call.

Exercise 4

[**Mutable Lambda — Counter**] Build a **reusable counter factory** using mutable lambdas.

- Write a function `function<int()> makeCounter(int start, int step)` that returns a mutable lambda. Each call to the returned lambda increments the captured counter by `step` and returns the new value.
- In `main()`, create three counters: `byOne` (start 0, step 1), `byFive` (start 0, step 5), and `countdown` (start 20, step −3).

- Call each counter five times and print its output.
- Show that the counters are **independent**: advancing `byOne` does not affect `byFive`.

Exercise 5

[**Map with `std::transform`**] Apply map operations to a collection of student records.

- Define a `struct Student { string name; double score; }`.
- Given a `vector<Student>` of at least six students with scores between 0 and 100:
  i. Use `transform` to produce a `vector<string>` of grade letters: A (≥ 80), B (≥ 65), C (≥ 50), F (otherwise).
  ii. Use `transform` to produce a scaled score vector where each score is curved by adding 5 points (cap at 100).
  iii. Use the binary form of `transform` to produce a `vector<string>` combining name and grade: `"Alice: A"`, `"Bob: C"`, etc.
- Print all three result vectors.

Exercise 6

[**Filter with `copy_if` and `remove_if`**] Process a log of server events using filter operations.

- Define `struct Event { string level; string message; int code; }`. Populate a `vector<Event>` with at least eight entries mixing levels `"INFO"`, `"WARN"`, and `"ERROR"`.
- Use `copy_if` to extract only `"ERROR"` events into a new vector and print them.
- Use the erase-remove idiom (`remove_if` + `erase`) on the original vector to delete all `"INFO"` entries in-place.
- Print the remaining events after the in-place removal.
- Write a reusable generic helper: `template<typename T, typename Pred> vector<T> filter(const vector<T>& v, Pred pred)` and demonstrate it on both the Event vector and a plain `vector<int>`.

Exercise 7

[**Reduce with `std::accumulate`**] Compute descriptive statistics for a dataset using `accumulate` only (no manual loops allowed).

- Given `vector<double> data = {4, 8, 15, 16, 23, 42, 7, 3, 19, 11}`, use `accumulate` to compute:
  i. Sum and mean.
  ii. Minimum and maximum (custom binary op).
  iii. Sum of squares.
  iv. Variance: $\sigma^2 = \frac{1}{n}\sum(x_i - \mu)^2$.
- Print all five statistics with two decimal places.
- Repeat the sum and mean using `std::reduce` (C++17) and verify the results match.

Exercise 8

[**Map-Filter-Reduce Pipeline**] Build a complete data pipeline for an online shop's order database.

- Define:

```
struct Order {
    int    id;
    string customer;
    double amount;
    bool   isPaid;
};
```

  Populate a `vector<Order>` with at least eight orders, some paid and some unpaid, with varying amounts.

- Pipeline stage 1 — **Filter**: keep only paid orders with `amount > 100`.
- Pipeline stage 2 — **Map**: apply a 10 % loyalty discount; produce a `vector<double>` of discounted amounts.
- Pipeline stage 3 — **Reduce**: compute total discounted revenue and the count of qualifying orders.
- Print a summary: number of qualifying orders and total discounted revenue, formatted to two decimal places.
- Encapsulate the three stages in named lambdas stored as `std::function` objects, so the pipeline reads clearly in `main()`.

Exercise 9

[**`std::bind` and Partial Application**] Create a configurable **text formatter** using `std::bind`.

- Write a free function:

```
string formatMessage(const string& prefix,
                     const string& msg,
                     const string& suffix);
```

  that returns `prefix + msg + suffix`.

- Use `std::bind` to create three specialised formatters:
  i. `makeInfo` — prefix `"[INFO] "`, no suffix.
  ii. `makeError` — prefix `"[ERROR] "`, suffix `" !!!"`.
  iii. `makeDebug` — prefix `"[DEBUG] "`, suffix `" (line?)"`.

  Each formatter takes only the `msg` argument.

- Store all three in a `vector<function<string(const string&)>>` and apply each to the message `"Connection timeout"`.
- Combine `bind` with `transform` to apply `makeError` to a `vector<string>` of five error messages and print all formatted strings.

Exercise 10

[**Function Composition and Recursive Lambda**] Implement a **generic function composition chain** and a **recursive lambda**.

- Write a variadic template `compose(f, g, ...)` that returns a lambda equivalent to $f \circ g \circ \ldots$, i.e., applies the functions right-to-left. Demonstrate the two-function version and the three-function version.
- Build a number-processing pipeline by composing:
  i. `negate` — multiplies by -1.
  ii. `addTen` — adds 10.
  iii. `square` — squares the value.

  Apply the composed function to the integers 1 through 5 using `transform` and print the results.
- Write a **recursive lambda** for Fibonacci using `std::function`:

```
function<long long(int)> fib;
fib = [&fib](int n) -> long long {
    // your implementation here
};
```

  Print `fib(n)` for $n = 0, 1, \ldots, 10$.

****